	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 10

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o brigada: 04

Semestre: 2027-I

Fecha de entrega: 06/11/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**

1.1. Planteamiento del problema 3

1.2. Motivación 3

1.3. Objetivos 3

2. Marco Teórico **3**

3. Desarrollo **5**

3.1. Actividad 1: Análisis de las respuestas de las LLMs 5

3.2. Actividad 2: Programa de validación defensiva en constructores 7

4. Resultados **8**

4.1. Resultados de la Práctica 8

5. Conclusiones **9**

6. Referencias **9**

1. Introducción

1.1 Planteamiento del problema

El problema abordado en esta práctica, por un lado, radica en analizar y contrastar las respuestas conceptuales generadas por diferentes modelos de lenguaje (LLM) al consultarles sobre la jerarquía de la clase Error y las excepciones más comunes en interfaces gráficas de usuario (GUI). Por otro lado, consiste en verificar por qué el lanzamiento y captura de excepciones dentro de los constructores constituye un mecanismo eficaz para prevenir la instanciación de objetos en estados inconsistentes, inválidos o corruptos en memoria.

1.2 Motivación

La gestión rigurosa de errores en tiempo de ejecución para construir aplicaciones estables. Comprender cómo abortar de manera controlada la creación de un objeto ante parámetros no adecuados para evitar la propagación de referencias defectuosas en el programa. Asimismo, examinar cómo la formulación, el nivel de detalle y las restricciones técnicas en los prompts determinan la exactitud y profundidad de las respuestas proporcionadas por herramientas de inteligencia artificial.

1.3 Objetivos

- Evaluar y comparar las respuestas conceptuales de distintas LLMs respecto a la clase Error y las excepciones en interfaces gráficas de Java.
- Analizar el comportamiento del constructor de una clase de dominio implementando validaciones de rangos numéricos, restricciones de dominio y transformaciones de tipos de datos.
- Comprobar mediante pruebas dirigidas y el uso de bloques try-catch-finally que el lanzamiento de excepciones durante la instanciación interrumpe el ciclo de vida del objeto.

2. Marco Teórico

En el lenguaje Java, el mecanismo de transferencia y control de anomalías en tiempo de ejecución se encuentra estructurado a partir de la jerarquía de la clase

Throwable, la cual representa la raíz común para cualquier entidad susceptible de ser lanzada mediante la instrucción **throw** y capturada a través de un bloque **catch** [1]. A partir de esta clase base se derivan dos ramas que separan los tipos de problemas que pueden manifestarse: la clase **Exception** y la clase **Error** [1].

La rama **Error** agrupa situaciones catastróficas y condiciones críticas provocadas por el entorno de ejecución o por la propia Máquina Virtual de Java (JVM), tales como el agotamiento de recursos o fallas de enlace binario. Dado que representan fallos que comprometen la estabilidad del sistema subyacente, la documentación oficial establece que las aplicaciones convencionales no deben intentar capturarlas mediante bloques catch ni recuperarse de ellas [1]. Dentro de esta familia destacan clases como **VirtualMachineError**, de la cual se desprenden **OutOfMemoryError**, cuando la memoria dinámica en el heap resulta insuficiente para alojar nuevos objetos, y **StackOverflowError**, provocada por el agotamiento de la pila de ejecución ante llamadas recursivas descontroladas [1]. Asimismo, la subclase **LinkageError** agrupa fallas como **NoClassDefFoundError**, originadas cuando una dependencia requerida durante la ejecución no puede localizarse o interpretarse en el classpath [1].

Por su parte, la clase **Exception** modela condiciones excepcionales derivadas de la lógica del programa o de datos imprevistos que una aplicación puede y debe gestionar. Esta categoría se subdivide en excepciones comprobadas (*checked*) y no comprobadas (*unchecked*) [2]. Las excepciones no comprobadas corresponden a la jerarquía de **RuntimeException** y a sus derivadas directas, tales como **NullPointerException**, **NumberFormatException** o **IllegalArgumentException** [2]. El compilador de Java no impone el requisito estricto de captura o declaración formal (*catch or declare requirement*) para las excepciones unchecked, asumiendo que representan errores lógicos que deben prevenirse mediante una correcta codificación o detectarse de inmediato. En contraste, cualquier subclase de Exception que no descienda de RuntimeException constituye una excepción comprobada, obligando a su tratamiento explícito en la firma del método o dentro de un bloque try-catch [2].

El constructor tiene como responsabilidad primordial establecer el estado inicial de una instancia y salvaguardar sus invariantes de clase. Cuando un parámetro de entrada vulnera las reglas de negocio o los límites operativos de la entidad, el constructor no puede recurrir a códigos de retorno para notificar la anomalía [1]. En este contexto, el lanzamiento deliberado de una excepción interrumpe de forma inmediata el flujo de construcción, abortando la finalización del operador de instanciación [2]. Esto garantiza que ninguna referencia apunte hacia un objeto parcialmente configurado en la memoria dinámica [1]. El bloque de liberación determinista **finally** asegura la inspección del entorno tras el fallo, permitiendo confirmar que la variable receptora permanece en estado nulo si el constructor no culminó exitosamente su secuencia [2].

3. Desarrollo

3.1 Actividad 1: Análisis de las respuestas de las LLMs

La primera actividad consistió en formular consultas conceptuales a diferentes modelos de lenguaje sobre la clase Error y las excepciones más frecuentes en interfaces gráficas de usuario, analizando la precisión técnica de las soluciones generadas a partir de las instrucciones proporcionadas por cada integrante del equipo.

Análisis de la Respuesta 1

La primera consulta se planteó mediante dos mensajes breves, solicitando primero las clases de la jerarquía Error en Java y, en el segundo mensaje, las excepciones más comunes en interfaces gráficas.

En la primera pregunta, el modelo desglosó la estructura de java.lang.Error como una rama de fallos críticos derivados de Throwable, explicando las clases VirtualMachineError, LinkageError, AWTError, AssertionError y ThreadDeath. Acompañó la descripción con un esquema jerárquico textual y una tabla de diferencias operativas frente a la clase Exception. Para la segunda pregunta, la herramienta organizó las excepciones de interfaz gráfica a partir de sus causas de origen, clasificándolas en problemas de concurrencia por manipulación de vistas fuera del subproceso principal, anomalías en el ciclo de vida y estado de componentes no inicializados como NullPointerException e IndexOutOfBoundsException, fallos de conversión de formato tras la entrada de datos como NumberFormatException, y errores de renderizado o memoria gráfica. Como cierre, incorporó pautas preventivas enfocadas en la separación de hilos y la validación de datos.

La solución abarcó tanto el ecosistema de Java (Swing y JavaFX) como plataformas externas (.NET y Android). A pesar de incorporar clases ajenas a la biblioteca estándar de Java debido a la ambigüedad de la consulta, la respuesta aportó un esquema conceptual sólido al vincular el modelo de eventos y la regla de hilo único con los fallos más frecuentes en aplicaciones visuales.

Análisis de la Respuesta 2

La segunda consulta integró ambas peticiones en un único mensaje formulado con tono coloquial, solicitando de forma explícita que las explicaciones sobre excepciones en GUI y la jerarquía Error fueran didácticas y concisas.

El modelo abordó de forma balanceada ambas temáticas. En el apartado de GUI presentó una tabla descriptiva con NullPointerException, NumberFormatException,

IllegalArgumentException, IndexOutOfBoundsException, IOException, ClassCastException e IllegalStateException, aclarando de manera pertinente que no se trata de clases exclusivas de componentes visuales, sino de excepciones estándar de Java que emergen ante la interacción del usuario. Ilustró este comportamiento mediante un ejemplo de conversión numérica y estableció una distinción entre una excepción y una validación de reglas de negocio. En la sección de la jerarquía Error, describió la raíz Throwable y detalló que clases como OutOfMemoryError y StackOverflowError descienden específicamente de VirtualMachineError. Asimismo, añadió una tabla comparativa y un párrafo de síntesis.

La integración de las dos preguntas en un solo prompt evitó ambigüedades sobre el lenguaje de desarrollo y permitió a la herramienta proporcionar una explicación ordenada.

Análisis de la Respuesta 3

La tercera consulta implementó una formulación estructurada en dos mensajes consecutivos, condicionando las respuestas a un esquema de análisis basado en cuatro directrices: qué es, cómo funciona, por qué es importante y cómo se utiliza.

El modelo adaptó estrictamente su contenido a la estructura demandada. Para la jerarquía Error, explicó la instanciación y propagación en la pila de llamadas ante anomalías de la JVM, justificando que los errores actúan como un interruptor de emergencia y estableciendo como principio arquitectónico no capturarlos con bloques catch ni lanzarlos en la lógica de aplicación. Para las excepciones en GUI, situó el análisis en los entornos Swing y AWT, explicando cómo el modelo dirigido por eventos y el Event Dispatch Thread (EDT) absorben excepciones no controladas, manteniendo la ventana visible en un estado potencialmente inconsistente mientras la traza del error se imprime en la terminal. Además, detalló pautas de programación defensiva como la introspección de componentes mediante instanceof en manejadores de eventos polimórficos, la retroalimentación visual al usuario y la sincronización sobre el hilo de la interfaz.

La exigencia metodológica del prompt condujo a la profundidad técnica, vinculando la teoría de fallos con la arquitectura interna de ejecución de Java.

Observaciones

La solicitud directa y didáctica de la segunda consulta generó un material de consulta conciso y clasificado mediante tablas. Finalmente, el marco analítico de la tercera consulta obligó al modelo a explicar la mecánica interna del lenguaje y las repercusiones en tiempo de ejecución, confirmando que la incorporación de especificaciones

de formato y estándares de profundidad favorece la obtención de explicaciones técnicas.

3.2 Actividad 2: Programa de validación defensiva en constructores

La segunda actividad se centró en la implementación de un programa orientado a modelar la creación de empleados en un dominio hospitalario, aplicando verificaciones dentro del constructor para validar parámetros y evitar instancias con información no adecuada.

El diseño se estructuró a través de dos clases dentro del paquete de la práctica: la clase de entidad **Empleado** y la clase controladora **Main**. La clase Empleado encapsula como atributos privados el nombre completo, el rol asignado dentro de la institución, el número identificador único y la carga horaria mensual de trabajo, complementándose con métodos consultores de acceso y la sobrescritura de toString para representar su estado interno.

El constructor de Empleado centraliza la lógica de validación mediante tres filtros secuenciales antes de autorizar la asignación del estado del objeto: En el **primer filtro** verifica que las horas mensuales se ubiquen dentro del intervalo numérico de 1 a 80 horas; si el valor resulta menor a 1 o mayor a 80, la ejecución se suspende arrojando una excepción de tipo **IndexOutOfBoundsException** con una descripción del límite excedido. El **segundo filtro** evalúa la cadena correspondiente al rol profesional mediante comparaciones sin distinción de mayúsculas y minúsculas; si la descripción no coincide con "MEDICO" ni con "ENFERMERO", se detiene el proceso lanzando una **IllegalArgumentException**. Y el **tercer filtro** gestiona la lectura del identificador, el cual se recibe inicialmente como una cadena de texto; al procesarse mediante la conversión a un entero primitivo dentro de un bloque try interno, cualquier presencia de caracteres alfanuméricos no numéricos detona una **NumberFormatException**, la cual es interceptada para relanzar una **IllegalArgumentException** contextualizada que preserva la referencia al fallo original. Únicamente si el flujo de ejecución supera con éxito las tres barreras defensivas, los datos son asignados a los campos del objeto en memoria y el rol se almacena normalizado en mayúsculas.

La clase **Main** implementa la función de prueba a través de un método auxiliar parametrizado que ejecuta la instanciación dentro de un bloque de control **try-catch-finally**. La variable de referencia hacia el empleado se declara e inicializa en **null** previo al intento de construcción. En el bloque try se invoca la creación del objeto; en caso de culminar sin anomalías, se imprime el resultado formateado de la entidad. Los bloques catch se organizaron de forma especializada, capturando primero **IndexOutOfBoundsException** e **IllegalArgumentException**, además de mantener un

capturador genérico para la clase **Exception**. Por su parte, la cláusula **finally** imprime el valor alojado en la variable de referencia independientemente de la ruta tomada por el programa.

Para evidenciar el comportamiento del sistema, la clase controladora manejó cuatro escenarios de prueba con datos predefinidos: En el **primer escenario** se suministró una carga de 120 horas para un médico, detonando la interrupción en el primer filtro por límite numérico. En el **segundo escenario** se introdujo el rol de contador con 40 horas, provocando el lanzamiento de la excepción en la segunda barrera por tipo de uso no admitido. En el **tercer escenario** se ingresó un rol y horas correctas pero asignó una cadena alfanumérica al identificador, interrumpiendo el flujo en el tercer filtro durante la conversión numérica. Y en el **cuarto escenario** se proporcionaron datos que cumplieron la totalidad de las reglas de negocio, permitiendo que el constructor culminara y que la referencia en memoria asociara formalmente al nuevo objeto.

4. Resultados

4.1 Resultados de la Práctica

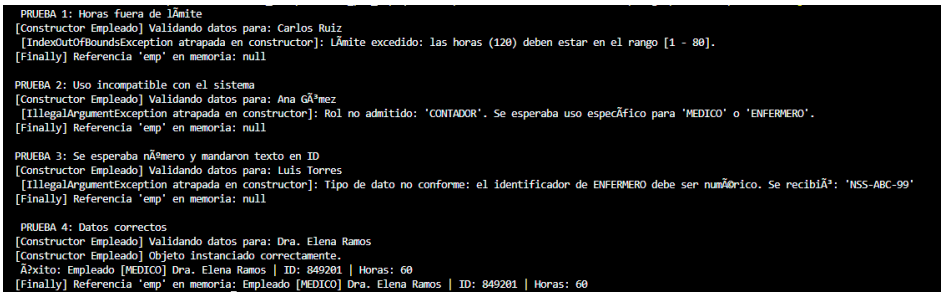


Figura 1: Compilación y ejecución completa del programa Main en terminal.

Figura 2: Diagrama de clases UML

Ver imagen 01_diagrama_clases_uml.png en GitHub

Figura 3: Diagrama de clases UML con cardinalidad

Ver imagen 01_diagrama_clases_uml_cardinalidad.png en GitHub

Figura 4: Diagrama de secuencia

Ver imagen 02_diagrama_secuencia.png en GitHub

Figura 5: Diagrama de flujo del constructor de Empleado

Ver imagen 03_flujo_empleado_constructor.png en GitHub

Figura 6: Diagrama de flujo de probarConstructor de en Main

Ver imagen 04_flujo_main_probarConstructor.png en GitHub

Figura 7: Diagrama de flujo del Main

Ver imagen 05_flujo_main_main.png en GitHub

5. Conclusiones

Los resultados obtenidos en las actividades evidencian que el lanzamiento de excepciones dentro de un constructor constituye el mecanismo formal y seguro en Java para preservar las invariantes de clase durante la instanciación. Dado que los constructores no retornan valores convencionales, las excepciones permiten suspender de manera inmediata la creación del objeto en el punto exacto donde se vulnera una regla de negocio. La presencia continua del valor null en la variable de referencia durante los tres primeros casos de prueba en la cláusula finally confirmó que la memoria queda protegida contra estados incompletos o referencias corruptas, habilitando la recolección de basura del objeto fallido. Asimismo, el contraste entre las consultas a los modelos de lenguaje demostró que la formulación de prompts con marcos analíticos estructurados permite obtener justificaciones técnicas profundas sobre el comportamiento de la máquina virtual y la arquitectura orientada a eventos.

6. Referencias

[1] Oracle, "Class Throwable," *Java Platform, Standard Edition & Java Development Kit Version 17 API Specification*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Throwable.html

[2] Oracle, "The Catch or Specify Requirement," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/essential/exceptions/catchOrDeclare.html